

V1 - Personal Finance Manager

Project Inception — Written Deliverable

Course: CSE 3311

Team: Team 6

Team Members: Ujjwal Sood, Hardik Kari, David Aviles, Ishika Sanghadia

Instructor: Christoph Csallner

1. Executive Summary

Personal Finance Manager is a computer program that was created with object-oriented principles in order to assist users in managing their finances. With Personal Finance Manager, users have access to a single interface in which they can enter all sorts of financial information such as income and expenses, budget for a month, saving plans, obligations and summaries of financial data. There is no need for a user to calculate their expenses and savings manually since all financial information is analyzed by the computer.

There are several features implemented into the application but one of the most important ones is "Can I afford it?" feature. This feature allows a user to enter the price of an item that he or she wants to buy and receive an estimation of how it will impact on the user's finances. The information such as available money, future payments, budgets and saving goals is considered before making an estimation. Each of the financial concepts used in the project is described with a separate class.

2. Problem Statement

The issue of managing money is rather complex, especially for students and young adults, who have just started to learn how to handle their income, expenses, savings, etc. While banking apps are capable of informing users about the transactions that were made, they are not always able to provide the necessary means to allow users to budget and see if they stick to the financial plan. A lot of people use spreadsheets or even hand-written notes to track their finances. This method may be problematic since it requires updating the information by hand and remembering all the expenses that will be occurring soon. Personal Finance Manager aims at solving this issue by integrating all these functions into one app.

3. Project Vision

Personal Finance Manager's vision is to develop an easy-to-use and realistic financial management tool that would help people to make wiser choices with respect to their finances. It

will turn basic financial data into valuable information by informing users of how much money they have at their disposal, how much they spend, how close they are to their budgets, and what progress they achieve in saving money.

It is important to emphasize that this system does not aim to substitute the professional financial advisory services; it will not offer any suggestions about investments. The purpose of the system is to simplify the process of financial management.

4. Project Goals and Objectives

Aims and Objectives

The main aim of the project will be to develop a dependable personal finance management program based on object-oriented software design. This system will give users the ability to log their incomes and expenditures and sort them into different categories. Also, users will be able to set monthly budgets and compare their spending with what was planned.

An additional objective will be to help users set saving goals and monitor their progress. A user can set, for instance, a target of saving \$1,000 for an emergency or \$2,000 for a new laptop. The progress will be logged, and the user will be updated about the money still required for reaching the target.

Finally, other functions such as recurring entries, financial alerts, monthly reports, and "Can I Afford It?" function will be included in the program.

5. Target Users

The main target group of Personal Finance Manager application is the students and young professionals and people who need an easy method of personal budget management. The students may find the tool helpful for managing their income from the job, paying rent, buying food, and paying for transport.

The young professionals may find the application helpful for managing their salaries, managing payments for monthly services, saving money, and making additional purchases.

6. Competitive Analysis

Personal Finance Manager faces three competitors that directly address the same audience as itself: YNAB (You Need A Budget), Rocket Money, and Monarch Money.

The first uses zero-based budgeting, meaning the user has to allocate every dollar to a particular budgeting category prior to the expense being made. This application allows users a high level of budgeting discipline and data analysis, however, has an inherent learning curve and is not free.

Rocket Money is aimed at analyzing recurring payments and negotiating them for the user, while providing some budgeting capabilities along with spending insight. This application has a free

version available, while its budgeting capabilities and negotiations are offered through the subscription.

Monarch Money allows users to manage their budget and bills. **Monarch Money** is aimed at couples and households who want to track spending and net worth together, and it connects an unlimited number of financial accounts, including investment and crypto accounts. It provides a polished, shared-budgeting experience but is priced as a paid subscription and is more feature-heavy than a student managing a single account typically needs.

Personal Finance Manager differentiates itself from all three by staying intentionally narrow and free: rather than replicating full net-worth tracking, investment monitoring, or bill negotiation, it focuses specifically on the "Can I Afford It?" affordability check a decision-support feature none of the three competitors offer in this form combined with straightforward budget and savings-goal tracking built for a single user rather than a household. Positioned on a simplicity vs. feature-depth map, YNAB and Monarch sit toward the "many features, steeper learning curve" end, Rocket Money sits in the middle, and Personal Finance Manager is designed to sit at the "easy to use, narrowly focused" end for someone just starting to manage their own money.

7. UI Screen Flow and Prototype

Personal Finance Manager will use the following core screens: a **Dashboard** showing available funds, recent transactions, and budget status at a glance; an **Add Transaction** screen for recording income or expenses; a **Budgets** screen listing each category's limit and usage; a **Savings Goals** screen showing progress toward each goal; a **Can I Afford It?** screen where the user enters a purchase price and receives a projected-impact result; and a **Reports** screen summarizing monthly financial activity.

The intended navigation flow is: the user lands on the Dashboard after logging in. From the Dashboard, tapping "+" opens Add Transaction; tapping a budget category opens the Budgets screen; tapping a savings goal opens Savings Goals; and a dedicated button opens the Can I Afford It? screen. Each of these screens has a back action that returns to the Dashboard, and the Reports screen is reachable from a menu accessible from any screen.

8. Core Use Cases

Based on the functional requirements above, Personal Finance Manager's core use cases are: Record Income, Record Expense, Create Budget, Track Budget Usage, Create Savings Goal, Track Savings Progress, Add Recurring Transaction, Check Affordability, Generate Monthly Report, and Receive Budget Alert.

Two of these use cases are detailed below as representative examples.

Use Case: Check Affordability

Actor: User. Precondition: The user has at least one account with recorded transactions.

Main flow: (1) The user opens the Can I Afford It? screen. (2) The user enters the price of a planned purchase. (3) The system retrieves the user's current available funds, upcoming recurring expenses, active budget limits, and savings goal targets. (4) The system calculates the user's projected remaining funds after the purchase. (5) The system displays the projected impact and highlights whether the purchase would interfere with any budget or savings goal.

Alternate flow: If the user has no recorded income or accounts, the system prompts the user to add an account first.

Use Case: Create Budget

Actor: User. Precondition: The user has at least one account.

Main flow: (1) The user selects "Create Budget" from the Budgets screen. (2) The user selects a spending category and enters a monthly limit. (3) The system saves the budget and begins tracking expenses recorded against that category. (4) The Budgets screen updates to show the new budget alongside its usage.

Alternate flow: If a budget already exists for the selected category, the system asks whether the user wants to edit the existing budget instead of creating a duplicate.

9. Core Features

The application will include an **income tracking system** that allows users to record different sources of income, such as salaries, freelance work, scholarships, or other payments. Users will be able to associate income with an account and a date so that the application can accurately calculate their available funds.

The **expense tracking system** will allow users to record purchases and assign them to categories such as food, housing, transportation, entertainment, shopping, and utilities. This information will be used throughout the application to calculate spending totals and budget usage.

The **budget management system** will allow users to establish spending limits for different categories. For example, a user could set a \$400 monthly food budget. As expenses are recorded, the system will calculate how much of that budget has been used and how much remains.

The **savings goal system** will allow users to create specific financial targets. Each goal will contain information such as the target amount, current saved amount, and target date. The application will calculate progress and show the user how much remains to reach the goal.

The application will also support **recurring transactions**. Users can define expenses such as rent, subscriptions, phone bills, or insurance payments that occur regularly. This information can

then be included when calculating projected available funds.

10. “Can I Afford It?” Feature

The “Can I Afford It?” feature is one of the main features that differentiates the application from a basic expense tracker. The user enters the price of an item they are considering purchasing. The system then evaluates the purchase using information already stored in the application.

The system considers the user's current available funds, upcoming recurring expenses, applicable budget limits, and savings goals. It then calculates the user's projected remaining funds after the purchase. The application presents the result in a simple format so the user can understand the potential impact of the purchase.

For example, if a user has \$1,500 available but has \$800 in upcoming required expenses and is trying to save \$500, the application can show that a large discretionary purchase may interfere with the user's financial plans. This feature provides decision support rather than professional financial advice.

11. Object-Oriented Design

Object-oriented programming will be a major part of the project's implementation. The application will represent important financial concepts as objects. This approach allows each class to have a specific responsibility and makes the application easier to maintain and expand.

The **User** class will represent the person using the application. A user can have multiple accounts, budgets, savings goals, and transactions.

The **Account** class will represent a financial account and maintain information about its balance and associated transactions.

The **Transaction** class will provide a common abstraction for financial activity. The system can use specialized classes such as **Income** and **Expense** to represent different types of transactions.

The **Budget** class will represent a spending limit and calculate how much of that budget has been used. The **SavingsGoal** class will manage a user's savings target and calculate progress toward that target.

Additional classes such as **RecurringTransaction**, **FinancialReport**, and **Notification** will provide support for recurring expenses, reports, and alerts.

12. Object-Oriented Programming Principles

The project will demonstrate several important object-oriented programming principles. **Encapsulation** will be used to protect the internal state of financial objects. For example, a Budget object should control how its spending information is modified rather than allowing unrelated parts of the application to directly change its internal data.

Inheritance can be used to allow Income and Expense objects to inherit common properties and behaviors from the Transaction class. This reduces duplicated code and establishes a logical relationship between related objects.

Polymorphism will allow different types of transactions or financial strategies to implement shared behaviors in different ways. **Abstraction** will be used to hide unnecessary implementation details and expose only the functionality required by other components.

The system will also use **composition**, where larger objects are built from related objects. For example, a User can contain multiple Accounts, Budgets, SavingsGoals, and Transactions.

13. Design Patterns

The project may use several design patterns where they provide a genuine architectural benefit. The **Strategy Pattern** could be used for different budgeting or affordability calculation strategies. This would allow the calculation method to be changed without modifying the rest of the application.

The **Observer Pattern** could be used for notifications. When a user's spending reaches a defined percentage of a budget, the relevant notification component could be informed automatically.

The **Repository Pattern** can be used to separate database operations from business logic. This means that the application can change its database implementation without requiring major changes to the financial domain classes.

14. System Architecture

The application will use a layered architecture consisting of a Presentation Layer, Application or Service Layer, Domain Layer, and Repository/Data Layer.

The **Presentation Layer** will be responsible for the user interface, including dashboards, forms, transaction screens, budget screens, and reports. The **Application Layer** will coordinate major user operations and connect the user interface with the underlying business logic.

The **Domain Layer** will contain the primary financial objects and rules, including Users, Accounts, Transactions, Budgets, and SavingsGoals. The **Repository/Data Layer** will handle storing and retrieving information.

Separating these responsibilities will reduce coupling and make the application easier to test, maintain, and modify.

15. Functional Requirements

The system shall allow users to create and manage financial accounts. Users shall be able to create, edit, and remove income and expense transactions. Each transaction shall be associated with an appropriate category.

The system shall allow users to create monthly budgets and calculate budget usage based on recorded transactions. Users shall also be able to create savings goals and monitor their progress.

The application shall support recurring income and expenses and display important financial information through a dashboard. The system shall generate alerts when users approach or exceed defined budget thresholds.

Finally, the system shall provide an affordability check and generate monthly financial summaries based on the user's stored financial information.

16. Non-Functional Requirements

The application should be easy for a new user to understand without requiring extensive instructions. Common operations such as adding a transaction, creating a budget, and viewing financial information should require minimal steps.

The application should also be reliable because incorrect financial calculations could lead to misleading results. Financial calculations should therefore be thoroughly tested, including edge cases such as zero balances, negative values, and budget limits.

Security and data integrity will also be important. Sensitive credentials should never be hard-coded into the application or committed to the repository. Financial values should be handled using an appropriate precision strategy rather than relying on unsafe floating-point calculations.

17. Risks and Mitigation

One major risk is incorrect financial calculations. This will be addressed through automated unit tests, boundary testing, and appropriate handling of monetary values.

Another risk is **scope creep**. The team could continue adding features such as investment tracking, bank integrations, artificial intelligence, or cryptocurrency tracking. To prevent the project from becoming too large, the team will prioritize the core requirements and only add optional features if sufficient time remains.

Usability is another potential risk. A technically functional application may still be difficult to use. The team will therefore create prototypes early and test common user workflows before the final implementation.

Using the probability $\times$ effect method from the Project Inception risk framework, the team's top risks are ranked below by risk exposure (RE):

1. Incorrect financial calculations — Probability: 40% (rounding and edge cases are easy to miss early on). Effect: about 10 extra developer hours to find and fix if not caught through testing. RE $\approx$ 4 hours.

2. Scope creep — Probability: 60% (student teams commonly add “just one more feature”). Effect: about 15 extra hours across the team if unchecked. RE $\approx$ 9 hours.

3. Poor usability — Probability: 35%. Effect: about 8 hours of rework to redesign confusing screens after feedback. RE $\approx$ 2.8 hours.

4. Uneven team member availability — Probability: 30% (a team member becoming unavailable due to coursework or other commitments). Effect: about 12 hours of redistributed work for the remaining team. RE $\approx$ 3.6 hours.

5. Underestimating the “Can I Afford It?” logic — Probability: 45% (combining budgets, recurring expenses, and savings goals into one calculation is more complex than it first appears). Effect: about 10 hours of rework if the initial design doesn't account for all these inputs correctly. RE $\approx$ 4.5 hours.

Ranked by RE, the team will prioritize scope discipline first, followed by getting the “Can I Afford It?” calculation logic right early, since these carry the highest exposure.

18. Development Plan

The first phase of development will focus on requirements, use cases, the object-oriented domain model, and repository setup. The second phase will implement accounts and basic transaction management.

The third phase will focus on budgets, categories, and the financial dashboard. The fourth phase will add savings goals, recurring transactions, and financial alerts.

The fifth phase will implement the “Can I Afford It?” feature and financial reports. The final phase will focus on integration testing, debugging, usability improvements, documentation, deployment, and preparation for the final project presentation.

19. Definition of Done

The initial version of Personal Finance Manager will be considered complete when users can create accounts and record income and expenses. Transactions must be categorized correctly and reflected in financial summaries.

Budgets and savings goals must calculate their values accurately. Recurring transactions and alerts must function according to the defined requirements. The “Can I Afford It?” feature must produce repeatable results using documented calculations.

The core object-oriented domain logic must have automated tests, and the application should be capable of running from a clean checkout of the project repository.

20. Conclusion

Personal Finance Manager provides a realistic software project that demonstrates object-oriented programming while solving an everyday problem. Rather than building a simple calculator or transaction list, the application combines multiple related financial concepts into a single system.

The project provides opportunities to demonstrate encapsulation, inheritance, polymorphism, abstraction, composition, design patterns, layered architecture, testing, data persistence, and user-interface development. The final application should provide users with a clear understanding of their financial activity while giving the project team a substantial and technically meaningful object-oriented software system to develop.

Link: - [soodujjwal/Team_6_Object_Oriented_Software](https://github.com/soodujjwal/Team_6_Object_Oriented_Software)